

Série 3 - Les structures séquentielles

Exercice 1

On considère, dans cet exercice, une liste L simplement chaînée d'entiers représentée par la structure vue dans le cours.

Pour favoriser la manipulation des listes chaînées, il est interdit d'utiliser les opérations de base vues dans le cours pour répondre aux questions suivantes.

1. Ecrire un module permettant de chercher le plus grand entier de la liste L et de retourner son adresse.
2. Ecrire une fonction récursive permettant de compter le nombre d'occurrences d'une valeur donnée x dans une liste L.
3. Ecrire un module permettant d'insérer un entier donné dans une liste triée par ordre croissant.
4. Ecrire un module permettant de supprimer de L la première occurrence d'une valeur donnée en paramètre.

Exercice 2

On s'intéresse dans cet exercice à la comparaison du contenu d'une pile et celui d'une file.

1. Donner une version itérative et une version récursive d'une fonction qui retourne vrai si les éléments de la pile de son sommet vers sa base, sont les mêmes que ceux de la file de sa tête vers sa queue et faux sinon.
2. En utilisant la fonction développée précédemment, écrire une fonction qui retourne vrai si les éléments de la pile de sa base vers son sommet, sont les mêmes que ceux de la file de sa tête vers sa queue et faux sinon.

Exercice 3

On se donne trois piles P1, P2 et P3, la pile P1 contient une suite de nombres entiers positifs. En n'utilisant que les trois piles données :

1. Ecrire une procédure permettant de déplacer les entiers de P1 dans P2 de façon à avoir dans P2 tous les nombres pairs au-dessous des nombres impairs (l'ordre d'apparition des éléments pairs pourra être inversé).
2. Ecrire une procédure permettant de copier dans P2 les nombres pairs contenus dans P1. Le contenu de P1 après exécution de l'algorithme doit être identique à celui avant exécution. Les nombres pairs dans P2 doivent être placés dans l'ordre où ils apparaissent dans P1.

Exercice 4

On désire automatiser le traitement des dossiers dans une assurance.

Chaque dossier est représenté par le nom, prénom de l'assuré, le matricule (unique) ainsi que le code de la maladie.

Les dossiers étant traités dans l'ordre FIFO, on les représente par une file. Nous n'avons aucune hypothèse sur le nombre de dossiers à traiter.

1. Donner les structures de données permettant de représenter ce contexte.
2. Sans utiliser de structure intermédiaire, écrire une fonction qui étant donnée la file, permet de retourner le nombre de dossiers à traiter.
3. Ecrire une procédure permettant de traiter tous les dossiers de code X en priorité, les autres dossiers doivent apparaître dans le même ordre dans la file résultat.

Exercice 5

Soit une file de processus (dont on ne connaît pas le nombre) en attente d'exécution. Chaque processus possède un code en nombre entier et une priorité 1 ou 2 selon que le processus est un processus primaire (priorité=1) ou secondaire (priorité=2).

1. Donner une représentation adéquate pour la file de processus.
2. En utilisant la fonction prédéfinie `ExecuteProcessus(CodeProc : Entier)`, écrire un algorithme qui permet d'exécuter en premier lieu tous les processus primaires dans leur ordre d'apparition dans la file, ensuite en deuxième lieu tous les processus secondaires dans l'ordre inverse de leur apparition.

Exercice 6

Les internautes utilisent un logiciel pour naviguer sur Internet. Parmi ses fonctions il y a celle qui permet de revenir sur une page déjà accédée. Cette fonction est activée à l'aide de l'icône représentant  la flèche

Pour réaliser cette fonctionnalité, le navigateur insert dans une **pile** l'adresse de chaque page accédée de sorte à pouvoir y accéder de nouveau au besoin. Les adresses des pages sont mémorisées selon l'ordre d'accès **du plus récent au moins récent**. Nous ne disposons d'aucune hypothèse sur le nombre d'adresses mémorisées.

Après un certain temps, cette pile peut contenir la même page plus d'une fois. Pour améliorer l'efficacité du navigateur, on se propose d'écrire une procédure appelée `enleverDoublon` qui enlève de la pile des adresses des pages, les inscriptions multiples les **moins** récentes d'une page. Le résultat obtenu après l'exécution de cette procédure sera une pile où chaque page n'apparaît qu'une et une seule fois, et les adresses des pages seront placées de la plus récente (au sommet de la pile) à la moins récente adresse.

1. Quelle représentation de la structure de donnée pile choisissez-vous? Justifier votre choix et rappeler la déclaration de cette représentation.
2. Développer la procédure `enleverDoublon`.

Exercice 7

Une expression est une suite de variables combinées par un ensemble d'opérateurs avec éventuellement un jeu de parenthèses ouvrantes ou fermantes. Une expression peut être notée sous plusieurs formes :

1. Expression complètement parenthésée
2. Expression préfixée
3. Expression postfixée
4. Expression infixée

ECP	EPre	EPost	EInf
$((A-B)+C)$	$+ - ABC$	$AB - C +$	$A - B + C$
$((A+B)-C)/D$	$/ - + ABCD$	$AB + C - D /$	$A + B - C / D$

Dans ce qui suit, on s'intéresse aux expressions arithmétiques décrites par des opérateurs binaires (+, -, *, /) et des opérandes identifiés par une lettre ou un chiffre.

1. Ecrire une procédure qui, étant donnée une expression arithmétique complètement parenthésée (sous forme de chaîne de caractères), la traduit en une expression arithmétique postfixée.
2. Ecrire une procédure permettant d'évaluer une expression arithmétique postfixée donnée. Il est possible d'utiliser les opérations suivantes : **oper2**(x, y, z) qui retourne le résultat de l'opération binaire y sur les opérandes x et z, dans cet ordre et **valeur**(x) qui retourne la valeur associée à la variable x.